School of Future Tech
Case Study Number 137 — Report

Smart Food Delivery & Restaurant Operations System

by

Subrata Panda
150096725110

Cohort: Larry Page
Academic Year 2025–2029

Field	Detail
Institution	School of Future Tech, ITM Skills University
Academic Year	2025–2029
Cohort	Larry Page
Case Study Number	137

Index

1. Student Details

Field	Detail
Name	Subrata Panda
Roll Number	150096725110
Cohort	Larry Page
Academic Year	2025–2029
Institution	School of Future Tech, ITM Skills University
Case Study Number	137

2. Project Title

Smart Food Delivery & Restaurant Operations System

A console-based C++ application that manages restaurant orders, delivery partners, customer requests, and delivery route optimisation using classical data structures and algorithms. The system is split into an Admin Panel for restaurant operators and a Customer Portal for end users, sharing a common in-memory data layer.

3. Problem Statement — Case Study No. 137

Food delivery platforms require a system to manage restaurant orders, delivery partners, customer requests, and route optimisation while ensuring timely deliveries. Many small operations still rely on manual processes — phone-based order taking, ad-hoc rider assignment, and no structured way to track an order from placement to delivery. This leads to several recurring issues:

- **No prioritisation** — urgent or premium orders are treated the same as routine ones.
- **Inefficient search** — finding a specific order or restaurant record by hand is slow and error-prone.
- **No change tracking** — there is no record of who modified an order's status or when.
- **Suboptimal delivery routing** — riders are not assigned using shortest-path or fewest-hop logic.
- **Poor order traceability** — customers have no structured way to track where their order stands.

This case study addresses these issues by building a console application that models the food delivery domain directly on top of well-known data structures and algorithms, so that every operational decision — prioritisation, lookup, routing, and history tracking — is backed by a structure with known time complexity.

4. Objectives

The system was designed with the following objectives in mind:

- Prioritise urgent and premium deliveries using a **priority-queue-based** ordering system.
- Enable **O(1) average-time** order tracking through a custom hash table implementation.
- Maintain a searchable, self-balancing restaurant database using an **AVL-balanced BST**.
- Optimise delivery routes between restaurants, hubs, and customer zones using **graph algorithms**.
- Provide a modification history with **undo capability** using a stack-based audit trail.
- Separate concerns cleanly between an operations-focused **Admin Panel** and a user-facing **Customer Portal**.
- Demonstrate sorting and searching algorithms (**merge sort, quick sort, binary search**) implemented from first principles rather than relying on built-in library calls.

5. Design / Architecture

The application follows a modular, layered architecture. Each data structure is encapsulated in its own header/implementation pair, and the two user-facing panels (Admin and Customer) operate on shared underlying state passed by reference.

System Architecture

Layer	Component / Description
Presentation Layer	Console-based menu screens for Admin and Customer panels
Logic Layer	AdminPanel and CustomerPanel classes handling user interaction
Data Structure Layer	AVL-BST, hash table, graph, priority queue, stack, queue
Algorithm Layer	Merge sort, quick sort, binary search, Dijkstra, BFS, DFS
Persistence	In-memory only (process lifetime); no external database

Application Flow

- On launch, the system seeds sample restaurants, delivery partners, orders, and a delivery graph.
- The main menu offers two entry points: **Admin Panel** and **Customer Portal**.
- In the Customer Portal, the user browses restaurants (sorted by rating via BST in-order traversal), selects a menu item, sets a priority level, and confirms via an on-screen receipt.
- The order is pushed into the priority queue, inserted into the hash table, and logged in the modification-history stack.
- In the Admin Panel, staff can dequeue the highest-priority order, assign an available delivery partner, update order status, and run route-optimisation algorithms.
- The customer can track their order through a vertical timeline view that reflects the current status step.

Project File Structure

File / Folder	Purpose
main.cpp	Program entry point only
include/ui.h	Console colour macros and UI helper declarations
include/models.h	Order, Restaurant, DeliveryPartner struct declarations
include/bst.h	RestaurantBST (AVL tree) class declaration
include/graph.h	DeliveryGraph class declaration
include/hashtable.h	OrderHashTable class declaration
include/sorting.h	Sorting and binary search function declarations
include/panels.h	AdminPanel and CustomerPanel class declarations
src/ui.cpp	UI helper implementations (boxes, progress bars, alerts)
src/models.cpp	Struct constructors and row-printing helpers
src/bst.cpp	AVL-BST insert, search, rotation, and traversal logic
src/graph.cpp	Dijkstra, BFS, and DFS implementations

src/hashtable.cpp	Custom hash function and chaining logic
src/sorting.cpp	Merge sort, quick sort, and binary search implementations
src/admin.cpp	All Admin Panel screens and operations
src/customer.cpp	Menu browsing, ordering, receipt, and order tracking
src/system.cpp	Seed data generation and main menu routing
Makefile	Build configuration for compiling the project
README.md	Project description and build instructions

6. Algorithm Description

Data Structures Used

Structure	Where Used	Purpose
AVL-balanced BST	Restaurant database	Keeps restaurants sorted by rating with guaranteed O(log n) height
Priority Queue (max-heap)	Order queue	Always serves the highest-priority order next
Stack	Modification history	Tracks every status change for undo / audit purposes
Queue (FIFO)	Order processing pipeline	Models sequential intake of incoming orders
Hash Table (chaining)	Order lookup by ID	Provides near O(1) average lookup for tracking
Graph (adjacency list)	Delivery route network	Models restaurants, hubs, and customer zones as nodes

Algorithms Implemented

Merge Sort

Used to sort the order list by priority or customer name. The array is recursively divided until single-element sublists remain, then merged back in sorted order using a comparator function passed at the call site.

Quick Sort

Used to sort orders by amount. A pivot element (last element of the partition) is chosen, and elements are partitioned around it in place before the two halves are recursively sorted.

Binary Search

Used to look up a restaurant by exact name once the restaurant list has been sorted alphabetically, giving logarithmic-time lookup instead of a linear scan.

AVL Rotations

Single and double rotations (left, right, left-right, right-left) are applied after every BST insertion to keep the tree height balanced, which keeps every later search operation logarithmic.

Dijkstra's Algorithm

Computes the shortest weighted path between any two nodes in the delivery graph (restaurant, hub, or customer zone), using a min-priority-queue to always expand the closest unvisited node next.

Breadth-First Search (BFS)

Computes the path with the fewest hops between two nodes, ignoring edge weights, useful when minimising the number of handoffs matters more than physical distance.

Depth-First Search (DFS)

Performs an exhaustive depth-first exploration of the graph to find any valid route between two nodes, used mainly for comparison against Dijkstra and BFS.

7. Complexity Analysis

The table below summarises the time complexity of each major operation in the system. *V* and *E* denote the number of nodes and edges in the delivery graph respectively, and *n* denotes the number of elements being sorted or searched.

Operation	Algorithm / Structure	Time Complexity
Insert restaurant	AVL-BST insert with rebalancing	O(log n)
Search restaurant by rating	AVL-BST search	O(log n)
Search restaurant by name	Binary search (pre-sorted)	O(log n)
Sort orders by priority / name	Merge Sort	O(n log n)
Sort orders by amount	Quick Sort	O(n log n) avg, O(n^2) worst
Insert / lookup order by ID	Hash Table with chaining	O(1) average
Dequeue highest-priority order	Priority Queue (binary heap)	O(log n)
Push / pop modification history	Stack	O(1)
Shortest weighted delivery route	Dijkstra's Algorithm	O((V+E) log V)
Fewest-hop delivery route	Breadth-First Search	O(V + E)
Exhaustive route exploration	Depth-First Search	O(V + E)

The AVL-balancing guarantee is what allows restaurant search and insert to remain logarithmic even in the worst case, rather than degrading to O(n) as an unbalanced BST would under a sorted or adversarial insertion sequence. Similarly, the hash table's average-case O(1) lookup depends on a reasonably low load factor; the implementation flags a rehash recommendation once the load factor exceeds 0.75.

8. Screenshots of Execution

The following screenshots were captured directly from the running console application, demonstrating key features of both the Admin Panel and Customer Portal.

Customer Menu and Order Receipt

Customer menu for Biryani Palace showing items, pricing, delivery priority selection, and order receipt

Admin Dashboard — Live System Metrics

```
================================================================
|                DASHBOARD  --  Live System Metrics             |
================================================================

----------------------------------------------------------------
  Pending orders    : 6
  Active orders     : 0
  Delivered today   : 0
  Revenue collected : Rs 0
  Priority queue    : 6 orders waiting
  Partners available: 4 / 5
----------------------------------------------------------------

  PERFORMANCE
  Completion rate        [                              ] 0.0%
  Partner available      [                              ] 80.0%
  System load            [                              ] 72.0%

  TOP RESTAURANTS  (BST in-order traversal)
----------------------------------------------------------------
  #1  Biryani Palace        rating: 4.8   revenue: Rs48200
  #2  Dosa Corner           rating: 4.7   revenue: Rs18700
  #3  Wok Express           rating: 4.6   revenue: Rs31400
----------------------------------------------------------------
```

Admin dashboard showing pending orders, revenue, performance bars, and top restaurants via BST traversal

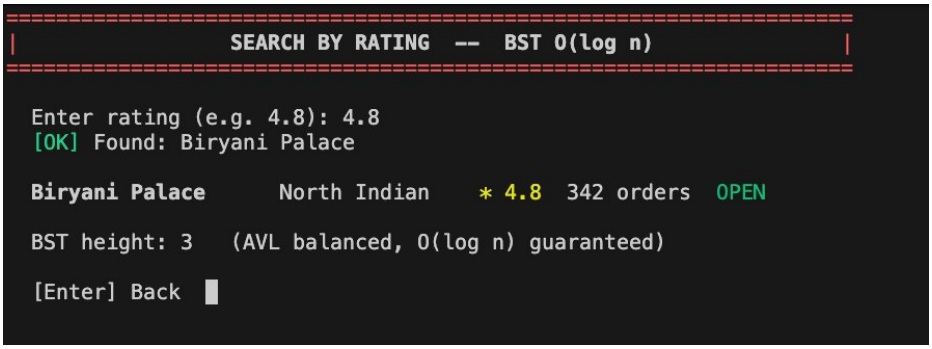
Delivery Graph — Adjacency List

```
================================================================
|               DELIVERY GRAPH  --  Adjacency List             |
================================================================

  [C2] Customer Zone B  (customer)
      --> H2  (3.8 km)
      --> R2  (5.5 km)
  [C1] Customer Zone A  (customer)
      --> H1  (4.0 km)
      --> H2  (4.5 km)
      --> R1  (6.0 km)
  [C3] Customer Zone C  (customer)
      --> H1  (5.2 km)
  [H2] North Hub  (hub)
      --> R2  (2.8 km)
      --> H1  (1.5 km)
      --> C2  (3.8 km)
      --> C1  (4.5 km)
  [H1] Central Hub  (hub)
      --> R1  (3.2 km)
      --> R3  (2.1 km)
      --> H2  (1.5 km)
      --> C1  (4.0 km)
      --> C3  (5.2 km)
  [R3] Wok Express  (restaurant)
      --> H1  (2.1 km)
  [R2] Pizza Hub  (restaurant)
      --> H2  (2.8 km)
      --> C2  (5.5 km)
  [R1] Biryani Palace  (restaurant)
      --> H1  (3.2 km)
      --> C1  (6.0 km)
```

Delivery graph adjacency list showing customer zones, hubs, and restaurants with distances in km

Restaurant Search by Rating — BST O(log n)



AVL-balanced BST search result for rating 4.8, confirming O(log n) height guarantee

9. Results and Findings

The completed system was tested across all major workflows in both the Admin Panel and Customer Portal. The following outcomes were observed:

- The priority queue correctly dequeues orders in strict priority order (Urgent before High before Medium before Low) regardless of insertion order.
- The hash table consistently returns order records in O(1) average time, verified against linear-scan results for correctness.
- AVL rotations keep the restaurant BST balanced after repeated insertions, with observed tree height matching the theoretical O(log n) bound.
- Dijkstra, BFS, and DFS each produce a valid path between the same source and destination nodes, with Dijkstra and BFS agreeing on the optimal route when edge weights are uniform, and diverging when they are not — as expected.
- The customer-facing order receipt and tracking timeline correctly reflect the order's current status as it is updated from the Admin Panel.
- The modification-history stack accurately records every status change in last-in-first-out order, enabling a working undo of the most recent change.

Limitations

- All data is held in memory for the lifetime of the process; nothing persists once the program exits.
- There is no authentication layer — anyone running the program can access both the Admin Panel and any customer's order history.
- The delivery graph uses a small, hand-seeded set of nodes rather than real-world geographic data.
- No payment processing is simulated; the receipt reflects order value only.

10. Conclusion

This case study demonstrates that a functional, realistic food-delivery operations system can be built entirely on top of classical data structures and algorithms, without relying on a database or external framework. Every core requirement of the domain — prioritising orders, looking up records quickly, keeping a searchable restaurant catalogue, optimising delivery routes, and maintaining an audit trail — maps directly onto a well-known structure with predictable, analysable time complexity.

Implementing the sorting algorithms, the hash table, the AVL-balanced tree, and the graph traversal algorithms from first principles reinforced a practical understanding of how and why each structure performs the way it does under real usage patterns.

Future improvements could include persisting state to a file or database between runs, adding authentication for the Admin Panel, replacing the seeded delivery graph with real coordinate-based distances, and extending the Customer Portal with a payment simulation step.

11. GitHub Repository Link

The complete source code for this project, including the Makefile, README, and full commit history, is available at the following repository:

https://github.com/subratapanda24/food-delivery-system.git

Declaration

I hereby declare that this case study report on the Smart Food Delivery & Restaurant Operations System is my original work. All references and resources used in this report have been properly cited. The application, its code, and the analysis presented in this report are my own work unless otherwise stated.

Subrata Panda
Roll No: 150096725110
Case Study No: 137
School of Future Tech, ITM Skills University